

Verified Collaborative Editing with Mergeable Replicated Data Types

Anonymous Authors

Abstract

Collaborative rich-text editors need three guarantees at once: replicas must converge, their states must implement an ordinary sequential editor, and their metadata must remain collectable. We develop a Lean 4 verification of Peritext over mergeable replicated data types (MRDTs). The proof includes three sequence kernels: an RGA baseline, an immutable-coordinate EmbedRGA, and a sided EmbedRGA with machine-checked Fugue and FugueMax ordering-policy theorems. It proves replication convergence, local minted-history refinement to independent sequential editor machines, and full merged-history theorems for all three kernels. Two collectors address different sources of growth. An asynchronous distributed collector removes commit history; a Peritext-specific collector removes dead text records, deletion evidence, and closed mark pairs while preserving future edits. Their composition refines the uncollected virtual-LCA semantics. A JavaScript implementation exercises the same designs. Repeated measurements show why both collectors and all three sequence baselines matter: on one long trace, EmbedRGA snapshots are 8.9 times smaller than RGA, while the sided variant pays additional metadata for a stronger non-interleaving policy.

Contents

1	The result	2
2	Why plain RGA is the motivating example	3
2.1	The tombstone baseline	3
3	Three verified sequence representations	4
3.1	EmbedRGA	4
3.2	SidedEmbedRGA and the Fugue policies	4
3.3	What is proved for all three	5
4	Peritext over the sided sequence	5
5	Datatype-state garbage collection	6
5.1	Three atomic collectors	7
5.2	Protocol refinement	7
6	Distributed commit-history garbage collection	8
7	Ordering anomalies and evidence	9
8	Implementation	9

9 Evaluation	10
9.1 Sequence kernels	10
9.2 Both collectors in Peritext	10
9.3 External plain-text systems	11
10 Evidence boundary and limitations	12
11 Reproduction	12

1 The result

The main result is an end-to-end semantic verification of a collaborative rich-text datatype. It is not one monolithic theorem. Figure 1 shows the evidence chain.

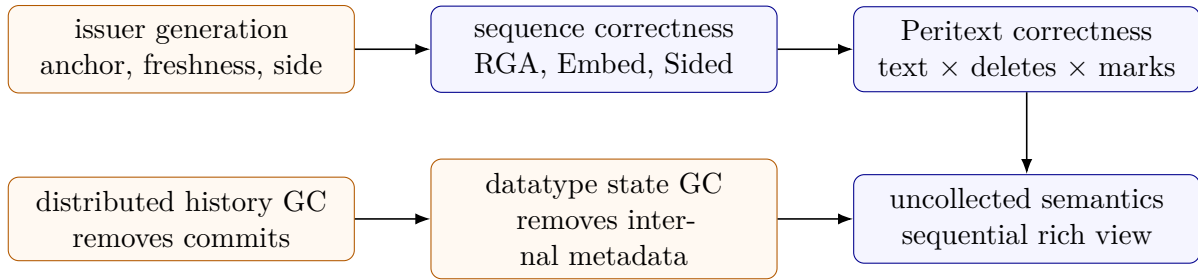


Figure 1: Two evidence paths meet at the user-visible result. The upper path proves the datatype; the lower path erases both collectors to that verified semantics. Blue boxes are correctness results and orange boxes are supplied protocol certificates.

The artifact establishes the following claims.

1. Every generation-certified ordinary or virtual-LCA execution of each sequence kernel is replication-aware linearizable.
2. Each legal sequential history refines an independent sequential specification. EmbedRGA and SidedEmbedRGA use list states; RGA uses a finite birth/grave reference with a list-valued sequence; Peritext uses a list-based rich editor and proves exact rendering.
3. Fugue has the stated forward non-interleaving property but not maximal backward non-interleaving. The FugueMax variant satisfies both directions and is replication-aware linearizable.
4. Sided Peritext has a physical state-GC transition system whose visible trace refines the uncollected semantic system. This protocol composes with the generic asynchronous commit-history collector.
5. The JavaScript runtime is a live, hand-written implementation validated by directed, randomized, snapshot, GC, and differential tests. It is not code extracted from Lean; an evidence manifest records the Lean package and correspondence status of each released datatype.

We use three evidence words consistently. A Lean declaration is *machine-checked*; a JavaScript test is *validated*; a benchmark number is *measured*.

2 Why plain RGA is the motivating example

An insert operation names an anchor and a globally fresh identifier:

$\text{ins}(t, a, x)$ “insert character x after identifier a ”.

A delete names the identifier to hide, $\text{del}(x)$. Identifier 0 is the permanent logical start anchor. A legal issuer may insert after a only when it has already observed a , and may delete only a known live identifier.

These premises are not cosmetic. The raw MRDT update is total and can store an insert record with an absent anchor. The user-facing theorem concerns traces minted by the editor protocol. The RGA generation contract records precisely that boundary:

$$\begin{aligned} \text{applicable}(\text{ins}(t, a, x), S) &\implies (a = 0 \vee a \in \text{born}(S)) \wedge t, x \text{ fresh}, \\ \text{applicable}(\text{del}(x), S) &\implies x \in \text{live}(S). \end{aligned}$$

The checked bridge turns these issuer-head checks into the history premise used by convergence and sequential refinement.

2.1 The tombstone baseline

The paper-level RGA state is a pair $S = (A, G)$:

$A \subseteq \mathbb{N}^3$, $(t, a, x) \in A$ records the birth of x after a ; $G \subseteq \mathbb{N}$ records deleted identifiers.

Both components grow monotonically.

$$\begin{aligned} \text{do}((A, G), \text{ins}(t, a, x)) &= (A \cup \{(t, a, x)\}, G), \\ \text{do}((A, G), \text{del}(x)) &= (A, G \cup \{x\}), \\ \text{mergeL}((A_0, G_0), (A_1, G_1), (A_2, G_2)) &= (A_0 \cup A_1 \cup A_2, G_0 \cup G_1 \cup G_2). \end{aligned}$$

To read the state, sort births by timestamp, splice each identifier after its anchor, and filter identifiers in G :

$$\text{seq}(A, G) = \text{filter}_{x \notin G}(\text{foldl insertAfter } [] \text{ sort}(A)).$$

These equations are the paper-level form of `Instances.RGA.RGAM` and `Instances.RGA.sequence`. The independent sequential machine `Instances.RGA.spec` stores finite birth and grave sets, with-out a replicated version DAG; `sequence` gives its intended list view.

The state is simple enough to explain both collectors. Deleting the entire document leaves all birth records and graves in the datatype state. Separately, the version DAG still contains every apply and merge commit. Commit-history GC cannot delete RGA tombstones because they live inside a retained state. Datatype-state GC cannot delete old versions because it sees only one materialization. Both are necessary.

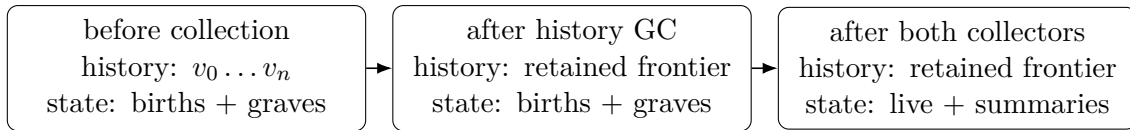


Figure 2: The collectors remove different metadata. Neither subsumes the other: the first changes the commit carrier; the second changes a retained version’s materialization. The left arrow is commit-history GC; the right arrow is datatype-state GC.

3 Three verified sequence representations

The three representations implement the same insert-after/delete interface, but they retain different information and offer different ordering policies:

Kernel	Position of a live descendant	What survives deletion of its anchor
RGA	identity edge to anchor a	birth of a and a grave marker
EmbedRGA	immutable anonymous path coordinate	path geometry; identity a may disappear
SidedEmbedRGA	path coordinate with an L/R band	path geometry and compact mint-policy summary

3.1 EmbedRGA

EmbedRGA assigns an immutable coordinate when an operation is minted. An insert carries a prefix π derived from its observed anchor and appends an ordered prefix-code word:

$$\text{coord}(\text{ins}(t, \pi, a, x)) = \pi \cdot \Gamma(t - a).$$

The materialized state is one list of (t, x, coord) records strictly sorted by coordinate. Insert is sorted insertion, delete filters one record, and ternary merge applies OR-set survival relative to the LCA before re-sorting. Because distinct honest births have distinct keys, sorted lists with the same record set are equal. This canonical representation is the core of the convergence proof.

The coordinate carries path geometry without retaining each deleted ancestor identity. This is more than a constant-factor encoding change after state GC: an ancestor spine can leave thousands of identity-bearing tombstones in RGA, whereas an embedded descendant needs only its path code.

3.2 SidedEmbedRGA and the Fugue policies

SidedEmbedRGA adds $d \in \{L, R\}$ to a minted insert and encodes it in a sided block:

$$\text{scoord}(\text{ins}(t, \pi, a, d, x)) = \pi \cdot \text{sBlock}_\Gamma(d, t - a).$$

The MRDT effect treats the side as immutable operation data. A generation policy decides which side and parent to mint from the issuer’s knowledge. This separation lets one proved kernel host several policies.

Definition 3.1 (non-interleaving). A local typing run is a sequence of causally consecutive inserts made by one replica. A merged read is forward non-interleaving when each forward run forms one interval. The maximal property also protects the paper’s backward-run cases, subject to its stated exception.

The Lean development proves:

$$\begin{aligned} \text{FugueReach} &\implies \text{ForwardNI}, \\ \text{Fugue} &\not\Rightarrow \text{MaximalBackwardNI}, \\ \text{FugueMaxReach} &\implies \text{ForwardNI} \wedge \text{BackwardNI} \wedge \text{MaximalNI}. \end{aligned}$$

The corresponding Lean results are:

```
fugue_forward_ni,    fugue_not_maximally_noninterleaving_backward,
fuguemax_forward_ni, fuguemax_backward_ni, fuguemax_maximally_noninterleaving.
```

The separate theorem `fuguemax_ra_linearizable` proves that the coordinate-level FugueMax policy model is RA-linearizable. That model’s `FMSig` is an internal proof signature, not another released datatype: it exposes proof-level coordinates and has no public `VerifiedMRDT` sequential package. The released datatype is `Instances.ProductionRGA.sided`; the FugueMax theorems justify its stronger ordering-policy design.

3.3 What is proved for all three

For each kernel, the repository packages a generation contract, ordinary and virtual-LCA convergence, an independent sequential machine, and the bridge from certified minting to sequential honesty. The capstones are `Instances.RGA.verified`, `Instances.ProductionRGA.embed`, and `Instances.ProductionRGA.sided`. Each is a public `VerifiedMRDT` package. The repository retains the separately named `replayEmbed` and `replaySided` packages for raw-fold proofs.

The sequential theorem matters independently of convergence. `EmbedRGA` and `SidedEmbedRGA` refine explicit list machines for a legal minted linear history. Each kernel also has the merged-history `VerifiedMRDT` theorem: each replicated version has an exact, interaction-respecting, legal sequential witness with the intended list view. `EmbedRGA` and `SidedEmbedRGA` use explicit `InteractionSpec` policies because universal commutation over malformed list states is not the client interaction relation. The executable `CRDTSig` contains no replay resolver; the old resolver is confined to one proof-local convergence route. The generation premise preserves the actual editor protocol rather than silently totalizing invalid anchor use as a no-op.

4 Peritext over the sided sequence

Peritext represents formatting as immutable mark events over a character sequence. The production-facing Lean state has three logical components:

$$S_{rich} = \underbrace{S_{text}}_{\text{sided insertion shadow}} \times \underbrace{D}_{\text{grow-only deleted character IDs}} \times \underbrace{M}_{\text{grow-only mark events}}.$$

Native text deletion is disabled. A user deletion adds the identifier to D , preserving the birth position needed to resolve mark boundaries. A mark records a globally fresh mark ID, type, value, start and end identifiers, and boundary sides.

At the paper level, a rich operation is one of

$$o ::= \text{textIns}(t, \pi, a, d, x) \mid \text{delete}(x) \mid \text{mark}(m, \text{kind}, \text{value}, a_s, d_s, a_e, d_e, \text{action}).$$

The update routes the operation to exactly one product component:

$$\begin{aligned} (T, D, M) + \text{textIns}(\dots) &= (T + \text{ins}(\dots), D, M), \\ (T, D, M) + \text{delete}(x) &= (T, D \cup \{x\}, M), \\ (T, D, M) + \text{mark}(m, \dots) &= (T, D, M \cup \{\text{mark}(m, \dots)\}). \end{aligned}$$

Ternary merge is the product of the sided-text merge and two set unions. This product structure explains convergence, but not client legality: the cross-component issuance policy supplies that missing relation.

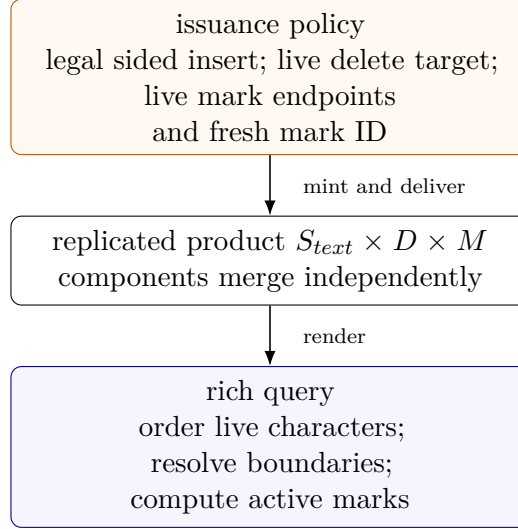


Figure 3: Peritext state and query pipeline. Text ordering, logical deletion, and mark evidence merge independently. The rich query is the only place that combines all three components.

The cross-component issuance policy in Figure 3 requires an insert to satisfy the sided sequence policy, a deletion to name a known live character, and a mark to name known live endpoints with a fresh mark ID. The theorem `coreHonest_of_mint` derives the sequence history fact needed by the product Join proof.

The independent rich sequential machine stores an ordinary list of character records, a deleted-ID set, and a mark-event set. The representation relation equates the sided MRDT projection with that list after removing the logical root, and equates both grow-only stores exactly. Consequently:

Theorem 4.1 (rich sequential correctness). *For every legal minted operation list ρ , the state obtained by folding the Sided Peritext MRDT is related to the state obtained by running the independent rich editor machine on ρ .* *rich_sequentially_correct*

The production signature narrows the query to rendered rich text without changing the operational carrier. `richVerified` transfers the merged-history legalization to this rendered query boundary. It proves query agreement for the document-order rich-text render, not only for the three raw components. `richReplayVerified` remains as the explicitly internal raw-fold package. The JavaScript export `peritext` selects `peritextSidedEmbedRGA`; the RGA and one-sided EmbedRGA variants remain available as controlled baselines.

5 Datatype-state garbage collection

Peritext cannot discard every deleted birth immediately. Mark endpoints and future Fugue minting can still depend on old order. A compact materialization therefore contains

$$C = \langle T, K, D, M, c \rangle,$$

where T is retained sided text, K is a finite LiveGap policy summary, D is retained deletion evidence, M is the mark list, and c is a stable Lamport cutoff. The cutoff has one interpretation:

$$\text{keep}(x) = \text{false} \implies x \leq c.$$

It is not a second tombstone set. It compactly separates collected old identifiers from future fresh identifiers. A new Lamport identifier must be strictly greater than c , so it is necessarily retained by the epoch policy.

The refinement relation connects this compact state to the uncollected rich state $F = (T_F, D_F, M_F)$. It existentially records a retention predicate k and Fugue knowledge K_F and requires

$$\begin{aligned} T_F &= \text{gFold}(K_F), & T &= \text{filter}_k(T_F), \\ \text{GapMapOK}(K_F, K), \neg k(x) & & \Rightarrow x \leq c, \\ \forall q. \text{query}(C, q) &= \text{render}(F, q). \end{aligned}$$

This is **StateGC.ProtocolRepresents**. Equality of current reads is only one field. The text projection and gap-map fields are what make subsequent insertion and merge provable.

5.1 Three atomic collectors

$$\begin{aligned} &\frac{\forall x. \neg k(x) \Rightarrow x \in D \quad \text{endpoints}(M) \subseteq k \quad K' \text{ exactly summarizes future mint choices}}{\langle T, K, D, M, c \rangle \rightsquigarrow \langle \text{filter}_k(T), K', D, M, \max(c, c') \rangle} \text{COLLECT-TEXT} \\ &\frac{}{\langle T, K, D, M, c \rangle \rightsquigarrow \langle T, K, D \cap \text{ids}(T), M, c \rangle} \text{TRIM-DELETES} \\ &\frac{m = \text{add} \quad r = \text{remove} \quad m < r \quad \alpha(m, r, M) \quad \beta(m, r, T)}{\langle T, K, D, M, c \rangle \rightsquigarrow \langle T, K, D, M \setminus \{m, r\}, c \rangle} \text{DROP-MARK-PAIR} \end{aligned}$$

Figure 4: The three Peritext state-GC steps. The α and β premises exclude delayed same-type marks and characters in the Lamport growth window; checked counterexamples show that both are necessary.

Each rule in Figure 4 preserves every rendered query: `collectText_query_preserved`, `trimDeleted_query_preserved`, and `dropMarkPair_query_preserved`. Query preservation alone is insufficient: the compact state must accept future operations and later merge with replicas collected in other epochs. The continuation proof uses three additional facts.

1. The LiveGap map reconstructs exactly the Fugue side, parent, and parent chain used to mint the next insert (`compactInsertOp_exact`).
2. Future Lamport IDs lie above the stable cutoff and therefore cannot be mistaken for collected IDs (`TextPlan.keeps_fresh`).
3. Independently collected branches translate to a common projection before ternary merge; compact merge equals that projection of semantic merge (`merge_text_after_epoch_translation`).

5.2 Protocol refinement

A physical version may have no materialization, but every replica head has one. Each materialized head carries a **Represents** witness linking compact text, gap summaries, cutoff, and all rich

queries to the semantic state. Collection and epoch reframing are silent. Ordinary apply/merge and virtual merge are visible.

The physical state contains one ghost semantic configuration for the proof and a partial materialization map:

$$P = \langle C_{sem}, \text{compact} : \text{Version} \rightarrow \text{CompactState} \rangle.$$

Every mapped version must represent its semantic state, and every replica head must be mapped. The transition labels are optional semantic labels:

$$\begin{array}{ll} P \xrightarrow{\tau} P' & : \text{collectText, trimDeleted, dropMarkPair, reframe, } C'_{sem} = C_{sem}, \\ P \xrightarrow{\ell} P' & : \text{ordinary, virtual, } C_{sem} \xrightarrow{\ell} \text{StepV } C'_{sem}. \end{array}$$

An apply label additionally requires its timestamp to lie above the compact head's stable cutoff. Fetch advances the Lamport clock before the runtime may mint that operation.

Theorem 5.1 (Peritext state-GC refinement). *Erase collection and reframe labels from any finite valid physical execution. The remaining labels form an execution of the uncollected virtual-LCA semantics, with the same rich queries at materialized heads.* **StateGC.Protocol.refines**

6 Distributed commit-history garbage collection

The second collector is generic to the MRDT runtime. Replica r stores only

$$L_r = \langle h_r, C_r \rangle, \quad h_r \in C_r,$$

where h_r is its head and C_r its locally held immutable commits. A remote fetch unions advertised commits; a separate head-sync action selects a held version. Evidence is derived from authored commits already in C_r .

The asynchronous transitions are:

$$\begin{array}{c} \frac{M = C_s}{W \longrightarrow W[d \mapsto \langle h_d, C_d \cup M \rangle]} \text{ FETCH} \\ \frac{v \in C_r}{W \longrightarrow W[r \mapsto \langle v, C_r \rangle]} \text{ HEAD-SYNC} \quad \frac{\text{Certificate}(\langle h_r, C_r \rangle, K)}{W \longrightarrow W[r \mapsto \langle h_r, K \rangle]} \text{ COLLECT.} \end{array}$$

Figure 5: Distributed commit-history transitions. Fetch corresponds to receiving immutable commits from a remote; it does not move the local head. Collection changes one replica and requires complete frontier evidence.

A local collection requires frontier evidence for every registered replica and a keep certificate that preserves head reachability and LCA answers. The compressed DAG connects retained versions according to original reachability; it need not retain the old root or every path node.

The collecting protocol refines the same asynchronous fetch/head-sync protocol with GC erased. State GC then composes directly with this refinement:

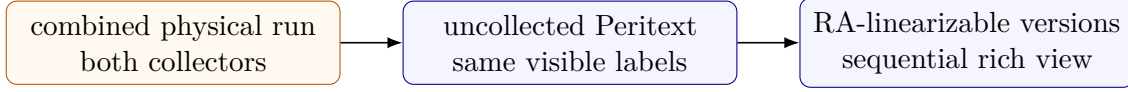


Figure 6: End-to-end semantic path for both collectors. The history collector is framework supplied; the state collector is Peritext supplied. Both arrows denote proved semantic implications: `CombinedSteps.refinesV` followed by the verified Peritext capstones. They do not denote implementation extraction.

`GC.CombinedSteps.refinesV` is the generic composition theorem. It avoids an intermediate global or stop-the-world semantics: distributed GC refines the no-GC asynchronous system directly.

7 Ordering anomalies and evidence

Finite anomaly fixtures are useful regression tests, but they are not a substitute for quantified intent theorems. Table 1 separates the two kinds of evidence.

Property or corpus	Scope	RGA	Embed RGA	Sided/ FugueMax
Sequential reference semantics	all legal sequential histories, Lean	proved	proved	proved
RA-linearizability	certified ordinary and virtual histories, Lean	proved	proved	proved
Forward non-interleaving	all Fugue-policy reachable states, Lean	—	—	proved
Maximal backward non-interleaving	policy-level quantified result, Lean	no claim	no claim	Fugue refuted; FugueMax proved
L1, L3a/b, L4, L5, L7–L13, L16, L17, L19	15 reviewed directed JS fixtures	baseline outputs	baseline outputs	all pass expected reads
L2, L6, L14, L15, L18, L20–L25	no current reviewed runtime fixture	untested	untested	untested

Table 1: Current anomaly evidence. “Untested” is not reported as a pass. The directed corpus validates implementation behavior; only the quantified Lean rows establish general semantic properties.

The L19 fixture is a backward-run case. Two replicas insert runs toward the front of a shared document. The reviewed sided implementations produce [50, 30, 10, 61, 41, 21, 1], keeping each local run contiguous. The stronger FugueMax theorem generalizes beyond this fixture. We do not claim an external Yjs, Automerge, or Loro anomaly result from the current harness because the repository does not contain reviewed adapters for the full anchored corpus.

8 Implementation

The JavaScript runtime provides content-addressed commits, replicas, fetch and head synchronization, three-way merge, snapshots, distributed frontier evidence, commit GC, and datatype state-GC hooks. Persistent maps and sets avoid copying full logical stores on each update. Binary snapshot codecs pack identifiers and coordinate symbols rather than shipping textual bit strings.

The runtime exposes six relevant datatypes: `rga`, `embedRGA`, `liveGapSidedEmbedRGA`, and `Peritext` built over each of those three kernels. The default `peritext` is the sided variant. Snapshot recovery means loading a native continuation snapshot into a fresh in-process state and materializing its visible document; it does not create a new logical replica or change replica identity.

The Lean code and JavaScript have different trust boundaries. Lean verifies mathematical definitions and refinements. JavaScript tests check that the implementation agrees with expected reads, a split-map oracle, randomized fork/join executions, snapshot round trips, frontier refusal, offline replica return, and collection invariants. Differential testing is a practical next step toward an extracted reference executable; no Lean-to-C-to-Wasm path is claimed here.

9 Evaluation

All reported cells are generated from checked-in drivers on Apple M4 Pro, 24 GB RAM, macOS arm64, and Node.js 26.3.1. Repeated reports use three isolated processes per cell. Drivers fail before reporting a cell if their semantic, convergence, snapshot, or scenario-specific GC gates fail.

9.1 Sequence kernels

Table 2 reports medians for four real sequential traces. *Recovery* loads the kernel’s continuation snapshot into a fresh state and materializes the visible sequence. Snapshot bytes are not a lower bound: each kernel has its own current codec.

Trace	Ops	Kernel	Apply (ms)	Snapshot (B)	Recovery (ms)
friendsforever	35,200	RGA	32.0	145,183	17.4
		Embed	20.5	59,064	13.4
		Sided	34.6	63,113	17.9
clownschool	43,500	RGA	19.1	138,358	16.8
		Embed	16.3	62,412	12.1
		Sided	25.8	62,912	16.0
seph-blog1	368,209	RGA	1,892.5	1,440,927	348.0
		Embed	1,795.1	161,046	47.4
		Sided	1,924.0	210,153	84.3
automerge-paper	260,978	RGA	2,031.9	1,182,880	267.8
		Embed	1,927.2	249,973	88.6
		Sided	2,083.2	323,888	138.6

Table 2: Repeated sequence-kernel comparison.

On `seph-blog1`, `EmbedRGA` uses 8.9 times fewer snapshot bytes than `RGA`; on `automerge-paper`, it uses 4.7 times fewer. `SidedEmbedRGA` uses 1–30% more snapshot space than `EmbedRGA` across these traces. On the two large traces, its apply time is 7–8% higher. This is the central design tradeoff: `EmbedRGA` minimizes current continuation state, while sided ordering pays metadata and compute for a stronger interleaving policy.

9.2 Both collectors in Peritext

Table 3 reports the combined-GC mode. Durable state is the continuation state needed for future editing, including the current materialization and GC summaries, not only visible characters.

Workload	Kernel	Time (s)	Durable state	Commits before → after
concurrent rich	RGA	13.56	205,426 B	4,108 → 1
	Embed	13.07	95,438 B	4,108 → 1
	Sided	14.53	212,685 B	4,108 → 1
format trace	RGA	35.98	204,936 B	26,753 → 1
	Embed	36.18	72,182 B	26,753 → 1
	Sided	36.63	203,219 B	26,753 → 1
mark churn	RGA	41.80	4,069 B	4,543 → 1
	Embed	42.21	1,014 B	4,543 → 1
	Sided	42.40	3,764 B	4,543 → 1
empty document	RGA	0.32	65 B	1,203 → 1
	Embed	0.32	9 B	1,203 → 1
	Sided	0.32	61 B	1,203 → 1

Table 3: Repeated Peritext evaluation with both state and commit-history GC.

The empty-document row is a sanity check, not an asymptotic theorem. On this settled workload, all character identities and marks disappear and history shrinks to one commit. The remaining 9–65 bytes are codec and continuation headers.

An isolated ancestor-spine workload explains why EmbedRGA remains useful even after both collectors. It performs 21,200 text operations and deletes 6,000 ancestor identities. RGA retains 150,561 bytes and all 6,000 identities; EmbedRGA retains 46,072 bytes and no deleted identity; SidedEmbedRGA retains 80,965 bytes and no deleted identity. All histories shrink to one commit. Anonymous path geometry can therefore be substantially smaller than an identity-bearing ancestor spine.

9.3 External plain-text systems

Table 4 gives one representative common-interface trace. These systems expose different continuation formats. Save bytes therefore describe each current native continuation snapshot; they are not semantic apples-to-apples lower bounds. We compare rich text only among the three Sal Peritext variants because the harness does not coerce external rich-text semantics into Peritext’s mark model.

System (seph-blog1)	Total (ms)	Save (B)	Recovery (ms)
Sal RGA	454.89	1,440,927	356.78
Sal EmbedRGA	367.41	161,046	52.67
Sal SidedEmbedRGA	508.94	210,153	85.23
Yjs	750.40	338,289	9.31
Automerge	10,018.46	205,250	443.01
Loro	303.07	336,808	0.23
list-positions	485.01	572,686	3.65

Table 4: Repeated plain-text comparison on a common sequential trace. Native save formats retain different auxiliary information.

The fast recovery values for Yjs and Loro reflect their native snapshot representations and optimized implementations. They do not establish the absence of ordering anomalies, and the current paper does not infer such a result from performance tests.

10 Evidence boundary and limitations

The current artifact proves the semantic kernels, virtual-LCA executions, issuance policies, sequential refinements, Peritext state GC, distributed commit GC, and their composition. It validates but does not verify the JavaScript transliteration. Same-replica crash recovery and durable replica identity are not yet a formal protocol. The reviewed JavaScript anomaly corpus contains 15 directed cases, not all labels L1–L25. External rich-text semantics are deliberately not compared.

The stable-cut theorem assumes Lamport-style monotone identifiers. A runtime must advance its local clock after fetch before minting. The compact LiveGap map is required for future Fugue decisions even when it is irrelevant to the current read. Removing a stable dead leaf without the required policy summary has a checked counterexample that changes a future read.

11 Reproduction

From the repository root, run `./scripts/check-mrtdt-refactor.sh` for the Lean and runtime gates and `./scripts/check-working-papers.sh` for both papers and their declaration ledger. Regenerate the measurements with `node benchmarks/tools/run-kernel-comparison.mjs 3` and `node benchmarks/tools/run-peritext-paper.mjs --repeat 3`.